Referencje i kopie — typy wbudowane

Obiekty mutowalne i niemutowalne

Obiekt **mutowalny** (ang. *mutable*) pozwala zmieniać swój stan po utworzeniu. Na przykład można zmienić element listy albo dodać wpis do słownika.

Obiektu **niemutowalnego** (ang. *immutable*) nie można w ten sposób zmienić. Liczby, napisy i krotki są niemutowalne. Operacja dająca inną wartość może powiązać zmienną z innym obiektem zamiast zmieniać dotychczasowy.

W obu przypadkach zmienne wskazują obiekty. Przypisanie nie ma osobnych reguł dla „typów prostych” i „złożonych”. Różne skutki wynikają z mutowalności i rodzaju operacji.

Przypisanie nie tworzy kopii

Jak działa instrukcja podstawienia w przypadku list?

In [1]:
```
lista1 = [1, 2, 3]
```

In [2]:
```
lista2 = lista1
```

Czy powstała kopia? Zmieńmy element przez drugą zmienną:

In [3]:
```
lista2[1] = 7
```

In [4]:
```
lista1[1]
```

Out[4]:
```
7
```

Zmiana jest widoczna również przez `lista1`: obie zmienne wskazują tę samą listę. Mówimy, że są **referencjami** (ang. *references*) do tego samego obiektu.

Operator `is` sprawdza, czy chodzi o ten sam obiekt. Operator `==` sprawdza równość według reguł danego typu.

In [5]:
```
lista1 is lista2
```

Out[5]:
```
True
```

Sprawdź:

- Wizualizacja działania referencji do list na stronie Python Tutor

Liczby i ponowne przypisanie

Liczba całkowita jest niemutowalna. Instrukcja `n += 5` w poniższym przykładzie wiąże nazwę `n` z obiektem reprezentującym wynik dodawania, zamiast zmieniać liczbę wskazywaną wcześniej.

In [6]:
```
n = 10
```

In [7]:
```
poprzednia = n
```

In [8]:
```
n += 5
```

In [9]:
```
n
```

Out[9]:
```
15
```

In [10]:
```
poprzednia
```

Out[10]:
```
10
```

Do porównywania wartości liczb i napisów używamy operatora `==`, a nie `is`.

Zmiana listy a przypisanie w funkcji

Argument funkcji zostaje powiązany z lokalnym parametrem. Przypisanie nowej listy do parametru nie zmienia powiązania zmiennej, której użyliśmy przy wywołaniu.

In [11]:
```
def przypisz_pusta(lista):
    lista = []
    return lista
```

In [12]:
```
oceny = [3, 4, 5]
```

In [13]:
```
nowa = przypisz_pusta(oceny)
```

In [14]:
```
oceny
```

Out[14]:
```
[3, 4, 5]
```

In [15]:
```
nowa
```

Out[15]:
```
[]
```

Modyfikacja wspólnej listy jest natomiast widoczna również poza funkcją:

In [16]:
```
def wyczysc(lista):
    lista.clear()
```

```
    return lista
```

In [17]:
```
wynik = wyczysc(oceny)
```

In [18]:
```
oceny
```

Out[18]:
```
[]
```

In [19]:
```
wynik is oceny
```

Out[19]:
```
True
```

Samo zwrócenie listy z funkcji również nie tworzy jej kopii.

Płytka kopia listy

Metoda `list.copy()` tworzy nową listę, której elementy wskazują te same obiekty co elementy oryginału. To **płytka kopia** (ang. *shallow copy*).

In [20]:
```
pierwsza = [1, 2, 3]
```

In [21]:
```
druga = pierwsza.copy()
```

In [22]:
```
druga is pierwsza
```

Out[22]:
```
False
```

In [23]:
```
druga[0] = 100
```

In [24]:
```
pierwsza
```

Out[24]:
```
[1, 2, 3]
```

In [25]:
```
druga
```

Out[25]:
```
[100, 2, 3]
```

Podobnie można utworzyć kopię listy za pomocą wycinka obejmującego całą listę:

In [26]:
```
kopia_wycinek = pierwsza[:]
```

In [27]:
```
kopia_wycinek is pierwsza
```

Out[27]:
```
False
```

In [28]:
```
kopia_wycinek == pierwsza
```

Out[28]:
```
True
```

Listy zagnieżdżone

Płytka kopia listy nie kopiuje list znajdujących się w jej elementach.

In [29]:
```
lista = [1, [2, 3]]
```

In [30]:
```
kopia = lista.copy()
```

In [31]:
```
kopia[1] is lista[1]
```

Out[31]:
```
True
```

In [32]:
```
kopia[0] = 100
```

In [33]:
```
kopia[1].append(4)
```

In [34]:
```
lista
```

Out[34]:
```
[1, [2, 3, 4]]
```

In [35]:
```
kopia
```

Out[35]:
```
[100, [2, 3, 4]]
```

Podmiana pierwszego elementu zmieniła tylko kopię. Dopisanie do listy zagnieżdżonej zmieniło jednak wspólny obiekt. Wycinek `lista[:]` dałby taką samą płytką kopię.

Głęboka kopia listy list

Jeżeli chcemy niezależnie zmieniać także listy zagnieżdżone, potrzebujemy kopii obu poziomów. Taką kopię nazywamy **głęboką** (ang. *deep copy*).

Dla listy, której elementami są listy liczb, możemy wykonać ją ręcznie:

In [36]:
```
wiersze = [[1, 2], [3, 4]]
```

In [37]:
```
kopie_wierszy = []
for wiersz in wiersze:
    kopie_wierszy.append(wiersz.copy())
```

In [38]:
```
kopie_wierszy is wiersze
```

Out[38]:
```
False
```

In [39]:
```
kopie_wierszy[0] is wiersze[0]
```

Out[39]:
```
False
```

In [40]:
```
kopie_wierszy[0][0] = 100
```

In [41]:
```
wiersze
```

Out[41]:
```
[[1, 2], [3, 4]]
```

In [42]:
```
kopie_wierszy
```

Out[42]:
```
[[100, 2], [3, 4]]
```

Kopiujemy listę zewnętrzną i każdy jej wiersz. Liczby mogą pozostać wspólne, bo są niemutowalne. Dla większej liczby mutowalnych poziomów trzeba byłoby skopiować także te poziomy. Ta pętla jest rozwiązaniem dla opisanej struktury, a nie uniwersalnym sposobem kopiowania dowolnych danych.

Słownik z listami jako wartościami

Metoda `dict.copy()` również tworzy płytką kopię. Powstaje nowy słownik, ale przechowywane listy pozostają wspólne.

In [43]:
```
oceny = {"Jan": [3, 4], "Anna": [5]}
```

In [44]:
```
kopia_ocen = oceny.copy()
```

In [45]:
```
kopia_ocen is oceny
```

Out[45]:
```
False
```

In [46]:
```
kopia_ocen["Jan"] is oceny["Jan"]
```

Out[46]:
```
True
```

In [47]:
```
kopia_ocen["Jan"].append(5)
```

In [48]:
```
oceny["Jan"]
```

Out[48]:
```
[3, 4, 5]
```

Niemutowalna krotka z mutowalnym elementem

Krotka jest niemutowalna: nie można podmienić jej elementu. Może jednak zawierać referencję do listy, którą nadal można zmieniać.

In [49]:
```
para = ("oceny", [3, 4])
```

In [50]:
```
para[1].append(5)
```

In [51]:
```
para
```

Out[51]:
```
('oceny', [3, 4, 5])
```

Nie zmieniliśmy tego, na jakie obiekty wskazują elementy krotki. Zmieniliśmy zawartość listy wskazywanej przez drugi element. Próba `para[1] = [1, 2]` zakończyłaby się natomiast błędem `TypeError`.

Powielanie listy operatorem `*`

Powielanie listy powiela referencje do jej elementów, a nie same obiekty.

In [52]:
```
wiersze = [[0, 0]] * 3
```

In [53]:
```
wiersze
```

Out[53]:
```
[[0, 0], [0, 0], [0, 0]]
```

In [54]:
```
wiersze[0] is wiersze[1]
```

Out[54]:
```
True
```

In [55]:
```
wiersze[0][0] = 9
```

In [56]:
```
wiersze
```

Out[56]:
```
[[9, 0], [9, 0], [9, 0]]
```

Każdy z trzech elementów wskazuje ten sam wiersz. Osobne wiersze można utworzyć na przykład wyrażeniem listowym:

In [57]:
```
niezalezne_wiersze = [[0, 0] for _ in range(3)]
```

In [58]:
```
niezalezne_wiersze[0] is niezalezne_wiersze[1]
```

Out[58]:
```
False
```

In [59]:
```
niezalezne_wiersze[0][0] = 9
```

In [60]:
```
niezalezne_wiersze
```

Out[60]:
```
[[9, 0], [0, 0], [0, 0]]
```